CAPSTONE ASSIGNMENT REPORT

Final Capstone Project Report

Social Network Explorer (SNE)

---

1. Introduction

The **Social Network Explorer (SNE)** is a data-structure-driven application designed to model and analyze relationships between users in a social network. It demonstrates how concepts like graphs, hashing, trees, and traversal algorithms can be applied to real-world systems such as social media platforms.

---

2. Objective

- To represent users and their connections using graph structures
- To perform efficient search and traversal operations
- To implement real-world features like friend suggestions and connectivity analysis
- To integrate multiple data structures into a single project

---

3. Problem Statement

Modern social networks involve millions of users connected through relationships. Efficiently storing, searching, and analyzing these connections requires optimized data structures and algorithms.

The project aims to:

- Model users as nodes
- Model friendships as edges
- Provide efficient querying and traversal

---

4. System Design

4.1 Data Structures Used

Graph (Core Structure)

- Represented using **Adjacency List**
- Each user → node
- Each friendship → edge

Hash Table

- Maps user ID → user data
- Provides **O(1)** average lookup

Queue

- Used in **BFS traversal**

Stack / Recursion

- Used in **DFS traversal**

Optional Extensions

- Trie for username search
- Priority Queue for suggestions

4.2 Architecture Overview

- **User Module** → Add/search users
- **Connection Module** → Add/remove friendships
- **Traversal Module** → BFS, DFS
- **Analysis Module** → Suggestions, connectivity

---

5. Features Implemented

5.1 Add User

- Create a new user node
- Store in hash table

5.2 Add Connection

- Add edge between two users
- Undirected graph

5.3 Display Network

- Show adjacency list

5.4 BFS Traversal

- Finds shortest path (unweighted)
- Explores level-wise

5.5 DFS Traversal

- Explores deep connections

5.6 Friend Suggestions

- Suggest friends-of-friends
- Avoid already connected users

5.7 Search User

- Fast lookup using hash table

---

6. Algorithms Used

6.1 Breadth First Search (BFS)

- Uses queue
- Time Complexity: **O(V + E)**

6.2 Depth First Search (DFS)

- Uses recursion/stack
- Time Complexity: **O(V + E)**

6.3 Hashing

- Average lookup: **O(1)**

---

7. Sample Working

Input

Users: A, B, C, D
Connections:

- A ↔ B
- A ↔ C
- B ↔ D

Graph Representation

A → B, C
B → A, D
C → A
D → B

BFS from A

A → B → C → D

DFS from A

A → B → D → C

Friend Suggestion for A

- Suggested: D (friend of B)

---

8. Complexity Analysis

Operation	Time Complexity
Add User	O(1)
Add Connection	O(1)
BFS / DFS	O(V + E)
Search User	O(1) avg

---

9. Applications

- Social media platforms
- Recommendation systems
- Networking analysis
- Fraud detection

---

10. Advantages

- Efficient data representation
- Fast search using hashing

- Scalable for large networks
- Demonstrates real-world system design

---

11. Limitations

- Does not handle weighted relationships
- No real-time database integration
- Limited UI (if console-based)

---

12. Future Enhancements

- Add weights (interaction strength)
- Implement shortest path (Dijkstra)
- Add GUI (React / Web App)
- Use database (MongoDB / SQL)
- Real-time updates

---

13. Conclusion

The Social Network Explorer successfully demonstrates how multiple data structures work together in a real-world scenario. It highlights the importance of graphs, traversal algorithms, and hashing in building scalable and efficient systems.

This project strengthens understanding of both theoretical and practical aspects of data structures.

---

14. References

- Data Structures textbooks
- Online documentation (DSA concepts)
- Classroom notes and experiments